

Reinforcement Learning-based Fault-Tolerant Control for Unmanned Aerial Vehicles

1st Guoqi Wang

National Engineering Research Center of Robot Visual Perception and Control Technology, School of Robotics
Hunan University
Changsha, China
wang_guoqi@foxmail.com

2nd Xudong Wang*

National Engineering Research Center of Robot Visual Perception and Control Technology, School of Robotics
Hunan University
Changsha, China
wang_xudong@foxmail.com

3rd Yang Li

State Key Laboratory of Advanced Design and Manufacturing Technology for Vehicle, College of Mechanical and Vehicle Engineering
Hunan University
Changsha, China
lyxc56@gmail.com

4th Ziwei Gao

National Engineering Research Center of Robot Visual Perception and Control Technology, School of Robotics
Hunan University
Changsha, China
2582171730@qq.com

5th Zhaoke Ning

School of Aeronautics and Astronautics
Sichuan University
Chengdu, China
zhaokening2018@163.com

Abstract—This paper focuses on the fault-tolerant control (FTC) problem for unmanned aerial vehicles (UAVs) subject to possible multiple actuator failures, which is a tough problem to solve with traditional FTC methods since they usually require accurate mathematical models. To address the limitation of traditional FTC methods, a model-free FTC approach is proposed based on reinforcement learning (RL). Subsequently, the proposed approach is applied to construct fault-tolerant controller for UAVs without any knowledge of the quadrotor dynamic information. Then, an end-to-end control policy that can tolerant actuator failures is obtained, which can map the state of the UAVs directly to the control commands of the four rotors after learning. Finally, the effectiveness of the proposed fault-tolerant approach is demonstrated by using the flexible modular quadrotor simulator.

Index Terms—Unmanned aerial vehicles, Fault-tolerant control, Reinforcement learning

I. INTRODUCTION

Unmanned aerial vehicles (UAVs) are widely used in numerous applications including remote sensing [1], agricultural [2], military [3] and rescue [4] due to its high flexible and lightweight design [5]. Meanwhile, with the rapid development of artificial intelligence technology such as smart sensors, communications and microcontrollers, the capabilities of UAVs have been enhanced. As such, scenarios and tasks for UAVs are increasingly complicated. Consequently, the structure of UAVs becomes more and more complex to adopt these challenging tasks. Besides, the UAVs are usually threatened by actuator failures due to inadequate maintenance,

long sailing time and various unknown disturbances. Note that, if any component of the UAVs experiences a minor fault, the safe operation of the UAVs cannot be guaranteed [6]. Therefore, the controller must be robust enough to cope with those disturbances and failures simultaneously. A valid way to enhance the system reliability and safety is by adopting fault-tolerant control (FTC) strategies [7].

Recently, the FTC issue for UAVs has garnered the interest of many scholars and a number of excellent FTC approaches have been proposed. Liu *et al.* [8] develop a fuzzy active disturbance rejection controller to ensure FTC and optimal performance control of UAVs under external disturbances and actuator failures. Chung *et al.* [9] propose a FTC method to cope with defective thrust systems in multirotor UAVs when significant degrees of failure occur in several defective motors by using the optimal control. Avram *et al.* [10] design an adaptive FTC approach without a fault diagnosis mechanism. The asymptotic stability can be guaranteed for altitude and attitude tracking errors by applying the proposed FTC algorithm. Alireza *et al.* [11] perform fault detection and isolation (FDI) research using an adaptive neural network structure. Then, the FDI signal is applied with a nonlinear dynamic inversion technique to compensate for actuator failures, thereby achieving active FTC of the UAVs. Shen *et al.* [12] carry out investigations on quadrotor UAVs with varying centers of gravity and unknown moments of inertia to propose a novel control scheme, which is adaptive learning and fault-tolerant.

While model-based controllers are commonly used for drone control, obtaining accurate models can be a complex, challenging, or even impossible task [13]. Due to the previously mentioned challenges, there is a significantly emerging interest in the development of data-driven FTC methods for

This work was supported by the National Natural Science Foundation of China (62203159), Natural Science Foundation of Hunan Province (2022JJ40097) and the Open Project of Key Laboratory of System Control and Information Processing of Ministry of Education (Scip202206);

UAVs in the context of big data [14]. Hua *et al.* [15] develop an FTC method to optimize system performance for feedback control systems experiencing faults with reinforcement learning (RL) participation. Zhang *et al.* [16] address the FTC for systems with multiple actuator failures based on RL and fuzzy models. A new fuzzy integral RL-based FTC algorithm that runs in real-time for the system with actuator failures is proposed, where the dynamic matrices may be partially unknown. Zhao *et al.* [17] obtain the optimal controller for each UAV via RL firstly. Then, based on this optimal controller, a data-driven fault-tolerant controller was constructed to constrain the effects of actuator failures. Fei *et al.* [18] concern with the multiple faults and attacks problem for UAVs. A FTC strategy was proposed using RL to assist the stabilizing controller that does not require fault diagnosis and detection mechanisms. Although in [19], RL was used to obtain a controller that can maintain the stability of the UAVs in the event of a single motor failure, while actuator partial loss of effectiveness (LOE) is more common in engineering practice.

Inspired by previous research, this paper considers the FTC of UAVs under multi-actuator failures where RL is used alone as a controller, and the main contributions are the following: (1) without combining traditional control methods, an end-to-end controller is learned directly; (2) position and attitude control are realized simultaneously, and system stability can be maintained in the presence of multiple actuator failures.

The remaining sections of the paper will be structured as follows. Section II describes the basic theories and models. The general overview of FTC strategies is presented in Section III. In Section IV, the performance and validity of proposed method is demonstrated through a series of experiments. Finally, a summary of this paper is provided in Section V.

II. PRELIMINARIES

A. Reinforcement Learning Theory

The RL-based UAV control framework is shown in Fig. 1. It is the basic principle of RL that learning through perpetual interaction between an agent and its surroundings while maximizing the cumulative rewards acquired through trial-and-error [20]. Further, the dynamics of systems and high-dimensional continuous spaces are better perceived and learned through the introduction of neural network (NN) [21]. This eliminates the need for accurate mathematical models, which are usually extremely complex.

The learning process of RL can be defined using a Markov decision process (MDP) [22]. Generally speaking, MDP is represented by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{T}, \gamma \rangle$, where $\mathcal{S}$ represents the states set of agent in the environment, $\mathcal{A}$ indicates the set of feasible actions that the agent may execute, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{R}$ denotes the reward obtained from the interaction between agent and environment, $\mathcal{T}(s_t, a_t, s_{t+1}) \sim p(s_{t+1} | s_t, a_t)$ describes the probability distribution of the agent transitioning state to s_{t+1} after taking action a_t in state s_t , γ stands for the discount factor with $\gamma \in (0, 1)$.

Specifically, at each time step t , the agent obtains state s_t feedback from the environment, which is used by the agent

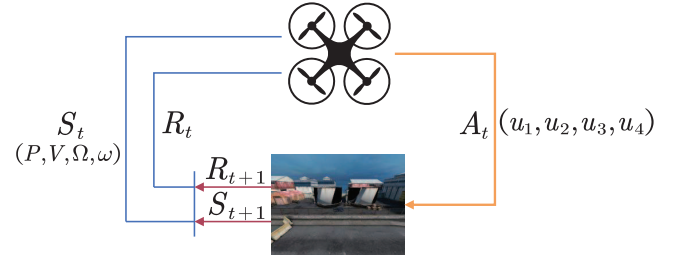


Fig. 1. RL-based UAV control framework.

to generate and execute an action a_t according to the policy $\pi(a_t | s_t)$. The environment then transitions to the subsequent state s_{t+1} according to the state transition probability $p(s_{t+1} | s_t, a_t)$, while an immediate reward r_t is feedback to the agent. This interaction process is repeated and generate a sequence ultimately:

$$\tau = \{s_0, a_0, r_0, s_1, \dots, s_t, a_t, r_t, s_{t+1}, \dots, s_T\},$$

with T is the maximum step size or time. The objective of RL is to find the optimal policy for maximizing the cumulative rewards obtained through the interaction.

B. Nominal Quadrotor Model

The quadrotor is considered a rigorous structure driven by four rotors, and the center of gravity coincides with its geometric center [23], [24]. The dynamics can be represented completely by the following Newton-Euler equation [19]:

$$\begin{cases} \dot{\mathbf{p}}_W = \mathbf{v}_W, \\ \dot{\mathbf{v}}_W = \mathbf{q}_W \odot \mathbf{c}_B - \mathbf{g}, \\ \dot{\mathbf{q}}_W = \frac{1}{2} \Lambda(\boldsymbol{\omega}_B) \cdot \mathbf{q}_W, \\ \dot{\boldsymbol{\omega}}_B = \mathbf{J}^{-1}(\boldsymbol{\eta} - \boldsymbol{\omega}_B \times \mathbf{J} \boldsymbol{\omega}_B), \end{cases}$$

where $\mathbf{p}_W$ and $\mathbf{v}_W$ represent the position and velocity vectors in the world frame W , respectively; $\mathbf{c}_B = (0, 0, c)^T$ denotes the mass-normalized thrust vector; $\mathbf{q}_W \odot \mathbf{c}_B$ represents the rotation of $\mathbf{c}_B$ by quaternion $\mathbf{q}_W$; $\mathbf{q}_W$ is a unit quaternion used to represent the orientation of the quadrotor; $\boldsymbol{\omega}_B$ describes the body rates in the body frame B ; $\mathbf{g} = [0, 0, -g_z]^T$ with $g_z = 9.81 \text{ m/s}^2$ is the gravity vector, and $\Lambda(\boldsymbol{\omega}_B)$ is a skew-symmetric matrix; $\mathbf{J}$, $\boldsymbol{\eta}$ stand for the diagonal inertia matrix and torque, respectively; c is obtained by converting the thrusts of single rotor $[f_1, f_2, f_3, f_4]$:

$$c = (f_1 + f_2 + f_3 + f_4)/m,$$

with m is the quadrotor's mass.

C. Actuator Failure Model

Previous work has shown that actuator failure is one of the major fault types of quadrotor. For instance, excessive deterioration or structural damage to the rotor or propeller may cause the UAV's actuator to malfunction, resulting in a partial loss of thrust generated by the corresponding rotor. Accordingly, actuator failure typically represented by constant partial LOE in the rotors is discussed in this summary. Throughout this

work, the actuator failure under consideration is modeled as follows, for $i = 1, \dots, 4$:

$$f_i^* = \alpha_i f_i,$$

where f_i represents the commanded rotor force of i th; f_i^* denotes the actual rotor force, and $\alpha_i \in [\underline{\alpha}, \bar{\alpha}]$ is an unknown constant characterizing the occurrence of a partial LOE fault in rotor i with $\underline{\alpha}$ being a known lower bound and $\bar{\alpha}$ being a known upper bound needed to maintain the controllability of the quadrotor. The scenario where $\alpha_i = 1$ indicates a healthy rotor, whereas $0 < \underline{\alpha} \leq \alpha_i \leq \bar{\alpha} < 1$ represents a broken rotor with partial LOE.

Furthermore, considering the situation where the actuator fails only during certain periods, the actual output force of the corresponding rotor can be expressed as

$$f_i^* = (1 - \beta_i(t \in [T_{begin}, T_{end}])\varrho_i)f_i, \quad (1)$$

with the fault time profile function $\beta_i(\cdot)$ is deemed a step function with an arbitrary failure interval $[T_{begin}, T_{end}]$, which can be described in the following form

$$\beta_i(t \in [T_{begin}, T_{end}]) = \begin{cases} 0, & \text{if } t \notin [T_{begin}, T_{end}], \\ 1, & \text{if } t \in [T_{begin}, T_{end}], \end{cases}$$

where T_{begin}, T_{end} represents the time when the fault started and ended, respectively. Additionally, the currently unknown failure coefficient for the i th actuator can be defined as

$$\varrho_i = 1 - \alpha_i.$$

III. FRAMEWORK PROCEDURE

A. Model-free Fault Tolerant Policy

The fault-tolerant controller is generated via training using the proximal policy optimization (PPO) algorithm, as the reference [25] shows a satisfactory performance and accuracy in the attitude control of UAVs. PPO can be used for environments with either discrete or continuous action spaces and is also an on-policy algorithm. An excess of policy updates can lead to a sudden collapse in performance for the typical policy gradient (PG). To solve this problem, some simple techniques are used in PPO to ensure that the pre and post-update strategies are not too different. PPO-Penalty and PPO-Clip are two primary variants of the PPO algorithm with PPO-Clip being widely adopted due to its implementation being simpler and more effective.

The parameters of the policy network are updated by PPO-Clip through the following formula:

$$\theta_{k+1} = \underset{\theta}{\operatorname{argmax}} E_{s, a \sim \pi_{\theta_k}} [\mathcal{L}^{CLIP}(s, a, \theta_k, \theta)].$$

Define $\rho_t(\theta)$ as the probability ratio and express it as

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)},$$

with π_{θ_k} and π_{θ} are the policies before and after the update, respectively.

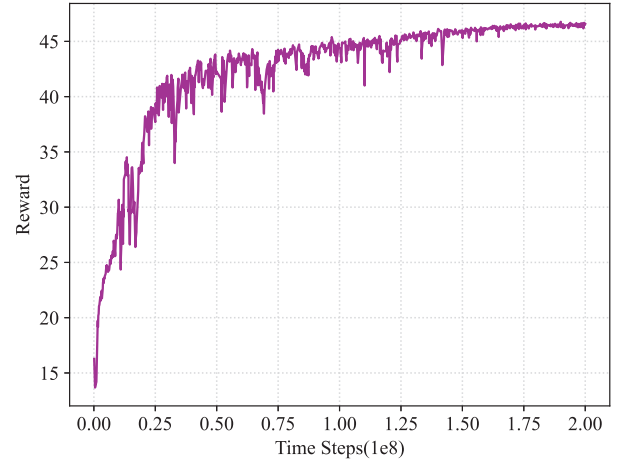


Fig. 2. The variation curve of episode cumulative reward in training process.

Then the objective $\mathcal{L}^{CLIP}$ is proposed by the following:

$$\mathcal{L}^{CLIP}(s, a, \theta_k, \theta) = \min \left(\rho_t(\theta) A_t^{\pi_{\theta_k}}, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t^{\pi_{\theta_k}} \right),$$

where $\text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)$ is the process of clipping $\rho_t(\theta)$ to the interval $(1 - \epsilon, 1 + \epsilon)$; ϵ denotes a hyperparameter regulating the magnitude of the clipping; $A_t^{\pi_{\theta_k}}$ stands for the advantage function, which generalized advantage estimation (GAE) method is used widely for calculation purposes. Firstly, the temporal difference error is described by $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$. Then GAE can be described as:

$$A_t^{GAE} = \delta_t + (\gamma\lambda) \delta_{t+1} + \dots + (\gamma\lambda)^l \delta_{t+l} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l},$$

with $\lambda \in [0, 1]$ is a hyperparameter introduced in GAE.

The loss function of critic network is computed through the mean square error shown below:

$$\mathcal{L}(\phi) = E \left[\left(V_{\phi}(s_t) - \hat{R}(s_t) \right)^2 \right],$$

where $\hat{R}(s_t)$ means the cumulative rewards starting from the state s_t ; ϕ is that critic network parameters can be updated using the following formula:

$$\phi_{k+1} = \underset{\phi}{\operatorname{argmin}} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \left(V_{\phi}(s_t) - \hat{R}_t \right)^2.$$

The FTC policy optimization is formulated with a standard RL problem. This involves a finite-horizon MDP with state space $\mathcal{S} \in \mathbb{R}^{12}$ and action space $\mathcal{A} \in \mathbb{R}^4$. At instant t , the state $s_t \in \mathcal{S}$ and action $a_t \in \mathcal{A}$ are as follows:

$$s_t = [\mathbf{p}, \mathbf{v}, \boldsymbol{\Omega}, \boldsymbol{\omega}], \\ a_t = [u_1, u_2, u_3, u_4],$$

with $\mathbf{p}, \mathbf{v}, \boldsymbol{\Omega}$ are denoted as the position, the Euler angle, and the velocity vectors in the world frame, respectively; $\boldsymbol{\omega}$

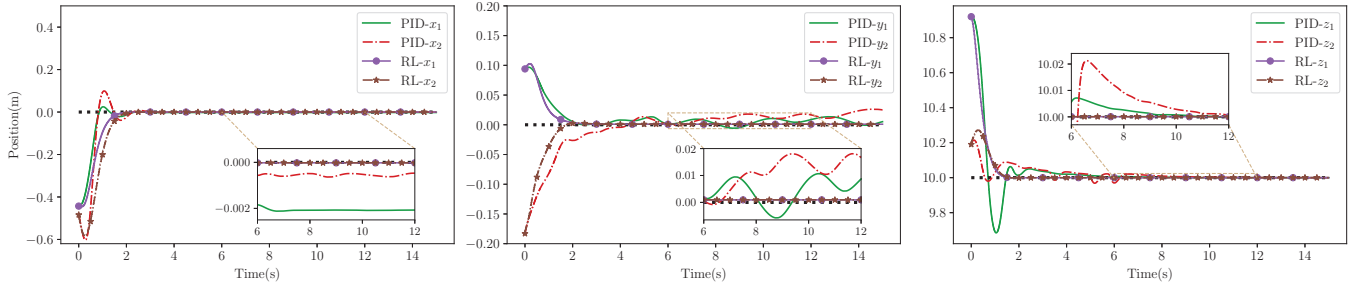


Fig. 3. The position control of the UAVs is shown when the actuator is not failing.

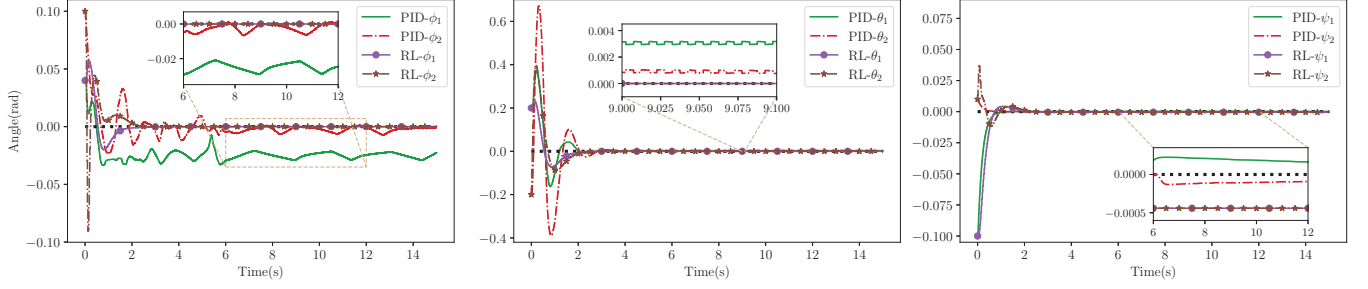


Fig. 4. The Euler angle control of the UAVs is shown when the actuator is not failing.

represents the angular velocity in the body coordinate system; $u_i \in [-1, 1], i = 1, 2, 3, 4$ are the throttle command for each rotor. After action a_t is taken, the reward r_t received is defined as:

$$r_t = w_1 e^{-t\|e_p\|} + w_2 e^{-t\|e_\Omega\|},$$

where w_1, w_2 are the weights of the corresponding reward term and t is the time; $\|e_p\|$ and $\|e_\Omega\|$ represent errors in position and Euler angle, respectively. The hyperparameters used are presented in Table I, and the configuration of the network for both the actor and critic are showed below:

- Actor

$$12 \times 128(\text{Tanh}) \times 128(\text{Tanh}) \times 128(\text{Tanh}) \times 128(\text{Tanh}) \times 4(\text{Tanh})$$

- Critic

$$12 \times 128(\text{Tanh}) \times 128(\text{Tanh}) \times 128(\text{Tanh}) \times 128(\text{Tanh}) \times 1$$

B. Problem Formulation

For simplicity, a hover task is assumed where the initial state of the quadrotor takes a random value within a certain range. Specifically speaking, $z \in [9, 11]$ and $x, y, \mathbf{v}, \mathbf{q} \in [-1, 1]$. Subsequently, the quadrotor is guided towards the target point by the RL controller, where the final desired state is set to (0, 0, 10, 0, 0, 0, 0, 0, 0, 0). It should be noted that the actuator is working normally throughout the training process. Then the performance of the learned controller is compared with the widely used proportional-integral-derivative (PID) controller under normal and fault conditions.

TABLE I
HYPERPARAMETERS USED IN PPO AND REWARD FUNCTION

	Actor	Critic
Optimizer	Adam	Adam
Learning rate	3e-4	3e-4
ϵ		0.2
λ		0.95
γ		0.99
Entropy Cost		1e-3
Timesteps		2e8
Batch Size		2.5e4
Reward function	$w_1 = 0.1, w_2 = 0.1$	

Under fault conditions, the failure coefficient is randomly generated from a uniform distribution. This coefficient is then multiplied by the thrusts generated by the drone as described in formula 1, the new thrusts will then actually act on the quadrotor. In our analysis, the range of values for the failure coefficient α_i is [0.7, 0.8], and the fault arises within a specific timeframe than a single incidence.

C. Training

Flightmare is a flexible modular quadrotor simulator including a configurable rendering engine and a flexible physics. Besides, it also provides users with a flexible interface for simulating different sensor data in various complex and near-reality environmental scenarios [19].

In the training process, each rollout has a maximum length of 250 samples which corresponds to 5 seconds, and each epoch has the maximum length of 25,000 samples. Due

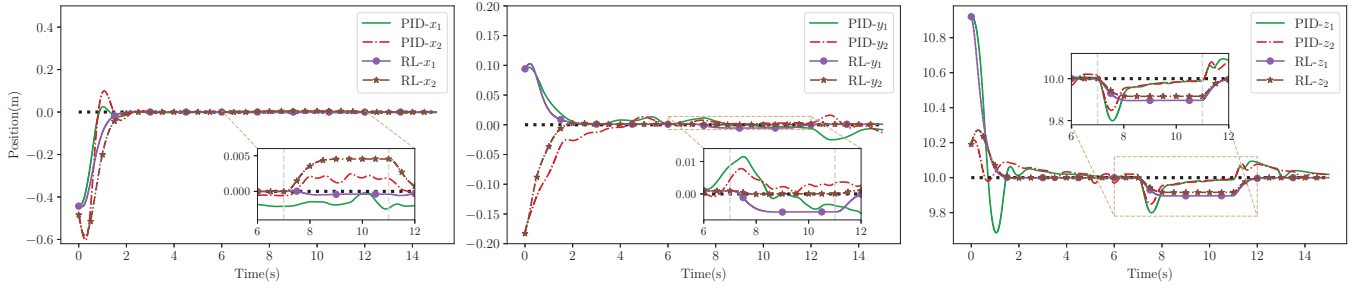


Fig. 5. The position control of the UAVs is shown when the actuator is exposed to partial LOE. The fault only occurs between 7 and 11 seconds and remains normal for the rest of the time.

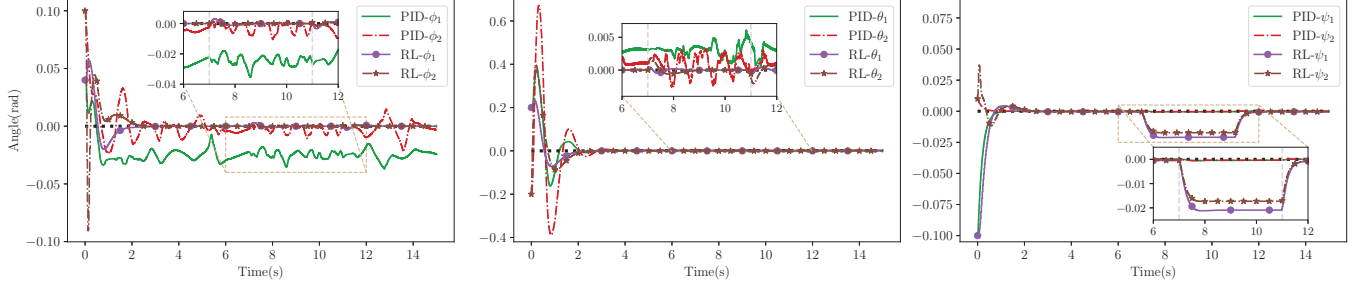


Fig. 6. The Euler angel control of the UAVs is shown when the actuator is exposed to partial LOE. The fault only occurs between 7 and 11 seconds and remains normal for the rest of the time.

to the parallel nature of Flightmare, 100 UAVs are used simultaneously to accelerate the training speed. The training process is accomplished on a desktop computer configured with 13th Gen Intel(R) Core(TM) i5-13400F CPU at up to 2.50GHz, employing the ubuntu22.04 operating system.

IV. SIMULATION RESULTS

The epoch in training lasted about 5 seconds, while the test lasted about 15 seconds. Fig. 2 shows the variation curve of episode cumulative reward in training process. As the training steps increase, valuable information is discovered by the policy resulting in increasing reward.

During the test stage, two groups of initial states are established to evaluate the validity of the RL and PID controllers. The corresponding initial state and fault coefficient settings are shown in Table II. Note that in the upcoming comparison test, it is assumed that the diagonal actuators suffer from LOE failures.

Fig. 3 and Fig. 4 show the response of the UAVs under the action of the RL and PID controllers when the actuator is not failing. Except for the yaw angle ψ , the RL controller achieves a faster response speed and less overshoot. In addition, the static error after stabilization is generally small when using RL controller. Only the steady-state error of yaw angle is larger than that of the PID controller, but the error is only $4e-4rad$, which is sufficient to meet the requirements.

Fig. 5 and Fig. 6 illustrate the corresponding curve of the UAVs under the action of two controllers when the diagonal actuator suffer a partial LOE failure. For positions x and y ,

TABLE II
THE SET OF INITIAL STATE AND FAULT COEFFICIENTS

state	1	2
x	-0.443	-0.484
y	0.094	-0.183
z	10.920	10.190
v_x	-0.049	-0.480
v_y	0.060	0.206
v_z	-0.171	0.420
ϕ	0.040	0.100
θ	0.200	-0.200
ψ	-0.100	0.010
α_1	0.714	0.727
α_3	0.736	0.792

both controllers are affected by the failure and neither has a distinct advantage over the other controllers. Looking immediately at altitude z , the overall variation is slightly greater for UAVs in the PID role. For the PID controller, the altitude of the UAVs dropped sharply after the fault and then slowly rose to the desired altitude. After the fault disappeared, the altitude of UAVs increased dramatically, followed by a slow descent. In contrast to the PID controller, the RL controller permitted the UAVs to descend relatively slowly by a certain altitude and stayed hovering after it suffered a fault. The UAVs returned to the target position and then remained stable without overshooting after the fault disappeared. As depicted in Fig. 6, the roll angle ϕ and pitch θ are almost unaffected by the failure under action with the RL controller, but this is not the case

for the PID controllers. Regrettably, the yaw angle is more affected than the PID controller under the action of the RL controller. Nevertheless, the deviation was only 0.02rad and the yaw angle quickly returned to its previous stable state after the fault. Overall, the RL controller is able to return the UAVs to its normal state faster than PID controller after experiencing a sudden failure.

V. CONCLUSIONS

In this article, the FTC problem for UAVs subject to partial LOE failure of the diagonal actuator is investigated using RL technique. Without combining any traditional FTC approaches, the fault-tolerant controller learns directly from the state data alone. Comparison with widely used PID controllers shows that the obtained controllers can perform at lower error levels and with greater stability. More importantly, the stability of UAVs can be maintained by the RL controller in the event of actuator failure. Once the fault has vanished, the UAVs can rapidly return to its previous state without overshooting. Future work is needed to explore the applications of more challenging tasks like trajectory tracking or multi-agent control, to improve the practicality of the RL-based FTC approach.

REFERENCES

- [1] N. M. Noor, A. Abdullah, and M. Hashim, "Remote sensing uav/drones and its applications for urban areas: a review," *IOP Conference Series: Earth and Environmental Science*, vol. 169, no. 1, p. 012003, 2018.
- [2] J. Kim, S. Kim, C. Ju, and H. I. Son, "Unmanned aerial vehicles in agriculture: A review of perspective of platform, control, and applications," *IEEE Access*, vol. 7, pp. 105 100–105 115, 2019.
- [3] R. Akter, M. Golam, V.-S. Doan, J.-M. Lee, and D.-S. Kim, "Iomt-net: Blockchain-integrated unauthorized uav localization using lightweight convolution neural network for internet of military things," *IEEE Internet of Things Journal*, vol. 10, no. 8, pp. 6634–6651, 2023.
- [4] M. Atif, R. Ahmad, W. Ahmad, L. Zhao, and J. J. P. C. Rodrigues, "Uav-assisted wireless localization for search and rescue," *IEEE Systems Journal*, vol. 15, no. 3, pp. 3261–3272, 2021.
- [5] L. Xiao, Z. Zhao, D. Cao, and J. Zu, "Composite nonlinear dynamic inversion control for quadrotor uav with multi-source disturbances and actuator faults," in *2022 41st Chinese Control Conference (CCC)*, 2022, pp. 386–391.
- [6] Z. Wang and L. Yao, "Fault diagnosis and fault-tolerant control for nonlinear uav system subject to sensor and actuator faults," in *2021 40th Chinese Control Conference (CCC)*. IEEE, 2021, pp. 4368–4373.
- [7] P. Cheng, Z. Gao, M. Qian, and J. Lin, "Active fault tolerant control design for uav using nonsingular fast terminal sliding mode approach," in *2018 Chinese Control And Decision Conference (CCDC)*. IEEE, 2018, pp. 292–297.
- [8] L. Xulin and G. Yuying, "Fault tolerant control of a quadrotor uav using control allocation," in *2018 Chinese Control And Decision Conference (CCDC)*. IEEE, 2018, pp. 1818–1824.
- [9] W. Chung and H. Son, "Fault-tolerant control of multirotor uavs by control variable elimination," *IEEE/ASME Transactions on Mechatronics*, vol. 25, no. 5, pp. 2513–2522, 2020.
- [10] R. C. Avram, X. Zhang, and J. Muse, "Nonlinear adaptive fault-tolerant quadrotor altitude and attitude tracking with multiple actuator faults," *IEEE Transactions on Control Systems Technology*, vol. 26, no. 2, pp. 701–707, 2018.
- [11] A. Abbaspour, K. K. Yen, P. Forouzaneshad, and A. Sargolzaei, "A neural adaptive approach for active fault-tolerant control design in uav," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 50, no. 9, pp. 3401–3411, 2020.
- [12] Z. Shen, L. Tan, S. Yu, and Y. Song, "Fault-tolerant adaptive learning control for quadrotor uavs with the time-varying cog and full-state constraints," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 12, pp. 5610–5622, 2021.
- [13] Y. Sohège, M. Quiñones-Grueiro, and G. Provan, "A novel hybrid approach for fault-tolerant control of uavs based on robust reinforcement learning," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 10719–10725.
- [14] L. Liu, S. Li, C. Han, and Y. Yang, "A data-driven fault tolerant control of singularly perturbed systems," in *2020 IEEE 9th Data Driven Control and Learning Systems Conference (DDCLS)*. IEEE, 2020, pp. 1410–1414.
- [15] C. Hua, L. Li, and S. X. Ding, "Reinforcement learning-aided performance-driven fault-tolerant control of feedback control systems," *IEEE Transactions on Automatic Control*, vol. 67, no. 6, pp. 3013–3020, 2022.
- [16] H. Zhang, K. Zhang, Y. Cai, and J. Han, "Adaptive fuzzy fault-tolerant tracking control for partially unknown systems with actuator faults via integral reinforcement learning method," *IEEE Transactions on Fuzzy Systems*, vol. 27, no. 10, pp. 1986–1998, 2019.
- [17] W. Zhao, H. Liu, and F. L. Lewis, "Data-driven fault-tolerant control for attitude synchronization of nonlinear quadrotors," *IEEE Transactions on Automatic Control*, vol. 66, no. 11, pp. 5584–5591, 2021.
- [18] F. Fei, Z. Tu, D. Xu, and X. Deng, "Learn-to-recover: Retrofitting uavs with reinforcement learning-assisted flight control under cyber-physical attacks," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 7358–7364.
- [19] S. Yunlong, N. Selim, K. Elia, L. Antonio, and S. Davide, "Flightmare: A flexible quadrotor simulator," *Conference on Robot Learning*, vol. abs/2009.00563, 2020.
- [20] W. Kaifang, L. Bo, G. Xiaoguang, H. Zijian, and Y. Zhipeng, "A learning-based flexible autonomous motion control method for uav in dynamic unknown environments," *Journal of Systems Engineering and Electronics*, vol. 32, no. 6, pp. 1490–1508, 2021.
- [21] T. Li, W. Bai, Q. Liu, Y. Long, and C. L. P. Chen, "Distributed fault-tolerant containment control protocols for the discrete-time multiagent systems via reinforcement learning method," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 3979–3991, 2023.
- [22] R. Sutton and A. Barto, "Reinforcement learning: An introduction," *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 1054–1054, 1998.
- [23] M. Faessler, A. Franchi, and D. Scaramuzza, "Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 620–626, 2018.
- [24] E. Kaufmann, A. Loquercio, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Deep drone acrobatics," *Robotics: Science and Systems (RSS)*, 2020.
- [25] W. Koch, R. Mancuso, R. West, and A. Bestavros, "Reinforcement learning for uav attitude control," *ACM Transactions on Cyber-Physical Systems*, vol. 3, no. 2, pp. 1–21, 2019.